

HAC (LAB_03) – U23AI060

Objective: To perform linear algebra operation to analyse the performance impact of scheduling clauses (static vs. dynamic) and the effect of memory access patterns (row-major vs. column-major) on parallel execution time.

Part 1: Vector-Vector Operations

Task: Implement a program that performs a **Dot Product** of two large vectors. Compare the execution time using static and dynamic scheduling.

Part 2: Matrix-Vector Multiplication (Row vs. Column)

Task: Observe how **spatial locality** and **synchronization** affect performance. C uses row-major ordering, making row-wise distribution much faster.

Program A: Row-Wise Distribution

In this version, each thread calculates a complete element of the result vector. No two threads write to the same memory location.

Program B: Column-Wise Distribution

In this version, threads work on columns. Because multiple threads will try to update the same $y[i]$ at the same time, we must use `#pragma omp critical` to prevent race conditions.

Part1 _ task _01

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000 // Large vector size

int main() {
    double *A, *B;
    double dot = 0.0;

    A = (double*)malloc(N * sizeof(double));
    B = (double*)malloc(N * sizeof(double));

    for (int i = 0; i < N; i++) {
        A[i] = 1.0;
        B[i] = 1.0;
    }

    /* ===== STATIC SCHEDULING ===== */
    dot = 0.0;
    double start_static = omp_get_wtime();

    #pragma omp parallel for reduction(+:dot) schedule(static)
    for (int i = 0; i < N; i++) {
        // Artificial load imbalance
        if (i % 1000 == 0) {
            for (int k = 0; k < 1000; k++)
                dot += A[i] * B[i];
        } else {
            dot += A[i] * B[i];
        }
    }

    double end_static = omp_get_wtime();

    printf("Static Scheduling:\n");
    printf("Dot Product = %f\n", dot);
    printf("Time Taken = %f seconds\n\n", end_static - start_static);

    /* ===== DYNAMIC SCHEDULING ===== */
    dot = 0.0;
    double start_dynamic = omp_get_wtime();
```

```

#pragma omp parallel for reduction(+:dot) schedule(dynamic, 100)
for (int i = 0; i < N; i++) {

    if (i % 1000 == 0) {
        for (int k = 0; k < 1000; k++)
            dot += A[i] * B[i];
    } else {
        dot += A[i] * B[i];
    }
}

double end_dynamic = omp_get_wtime();

printf("Dynamic Scheduling:\n");
printf("Dot Product = %f\n", dot);
printf("Time Taken = %f seconds\n", end_dynamic - start_dynamic);

free(A);
free(B);

return 0;
}

```

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/_HPC/LAB/03_lab_\$./p1

Static Scheduling:

Dot Product = 19990000.000000

Time Taken = 0.019425 seconds

Dynamic Scheduling:

Dot Product = 19990000.000000

Time Taken = 0.019781 seconds

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/_HPC/LAB/03_lab_\$

OBSERVATIONS :

In static scheduling, loop iterations are divided among threads before execution begins.

Threads processing computationally heavier iterations take longer,

while others remain idle at the implicit barrier, causing load imbalance and poor CPU utilization.

In dynamic scheduling, iterations are assigned to threads at runtime from a shared pool.

Threads that finish early receive additional work, resulting in better load balancing and reduced overall execution time despite minor scheduling overhead.

Part_02 :

03_lab_ > C 02.C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define N 2000
6
7  int main() {
8      static double A[N][N], x[N], y[N];
9
10     // Initialization
11     for (int i = 0; i < N; i++) {
12         x[i] = 1.0;
13         y[i] = 0.0;
14         for (int j = 0; j < N; j++)
15             A[i][j] = 1.0;
16     }
17
18     double start = omp_get_wtime();
19
20     #pragma omp parallel for schedule(static)
21     for (int i = 0; i < N; i++) {
22         double sum = 0.0;
23         for (int j = 0; j < N; j++) {
24             sum += A[i][j] * x[j];
25         }
26         y[i] = sum;
27     }
28
29     double end = omp_get_wtime();
30
31     printf("Row-wise Distribution Time = %f seconds\n", end - start);
32
33     return 0;
34 }
35
36 // Row-wise multiplication (why it is fast)
37 // In C, matrix data is stored row by row in memory.
38 // When we calculate row-wise, the program reads data that is already close together.
39 // Because of this:
40 // CPU cache works well
41 // Data comes quickly
42 // No extra waiting is needed
43 // Each thread writes to its own result value, so:
44 // No two threads fight
45 // No locks are needed
46 // That's why row-wise runs fast.

```

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/_HPC/LAB/03_lab_\$./p2
Row-wise Distribution Time = 0.010694 seconds

part_03

03_lab_ > C 03.C

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define N 2000
6
7  int main() {
8      static double A[N][N], x[N], y[N];
9
10     // Initialization
11     for (int i = 0; i < N; i++) {
12         x[i] = 1.0;
13         y[i] = 0.0;
14         for (int j = 0; j < N; j++)
15             A[i][j] = 1.0;
16     }
17
18     double start = omp_get_wtime();
19
20     #pragma omp parallel for schedule(static)
21     for (int j = 0; j < N; j++) {
22         for (int i = 0; i < N; i++) {
23             #pragma omp critical
24             {
25                 y[i] += A[i][j] * x[j];
26             }
27         }
28     }
29
30     double end = omp_get_wtime();
31
32     printf("Column-wise Distribution Time = %f seconds\n", end - start);
33
34     return 0;
35 }
36
37 // Column-wise multiplication (why it is slow)
38 // Column elements are not close together in memory.
39 // The program has to jump around in memory to read data.
40 // This causes:
41 // More cache misses
42 // More waiting for memory
43 // Also, many threads try to update the same result value.
44 // To stop wrong results, we use critical, which:
45 // Allows only one thread at a time
46 // Makes other threads wait
47 // This waiting makes the program slow.

```

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/_HPC/LAB/03_lab_\$./p3
Column-wise Distribution Time = 0.995008 seconds